StepModel v2 — Full Pipeline Documentation

> *Autonomous Penetration-Testing Step Planner*
> *Architecture: GNN Classifier → SFT LLM → GRPO RL Fine-tuning*

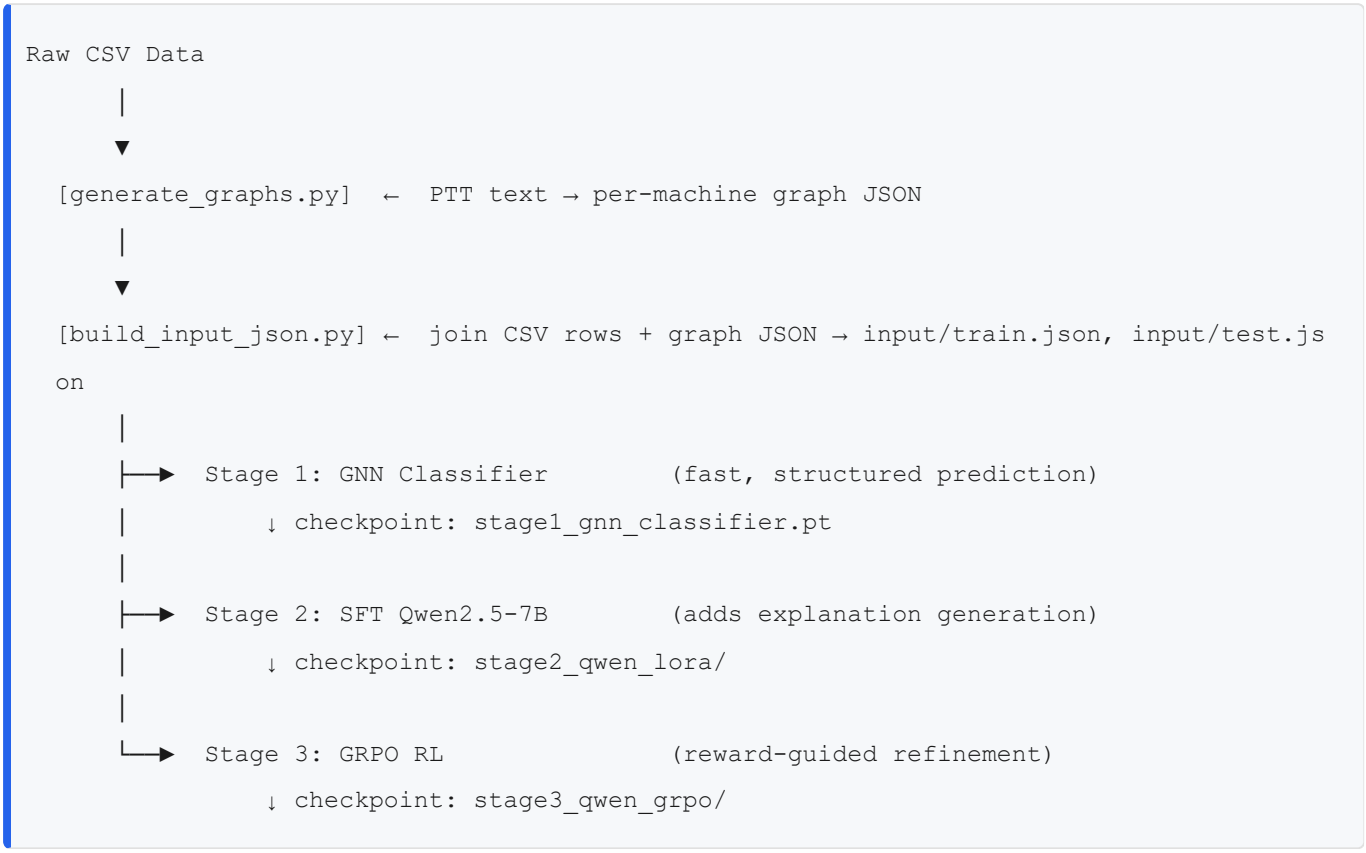
Table of Contents

Overview

StepModel v2 is a three-stage pipeline that predicts the **next step** a penetration tester should take, given the current state of a machine under test. At each decision point the system must:

1. Predict the **next step type** — one of 10 fixed tactical categories (e.g. "Exploit the selected exploitations", "Enumerate further on the X service...")
2. Predict the **MCP tools** to use — a multi-label subset of 11 tools (Nmap, Metasploit, Dirbuster, etc.)
3. Generate a **natural-language explanation** for the choice (Stages 2 and 3 only)

The pipeline trains three models in sequence, each building on the previous:



Dataset: 1,894 training rows across 325 machines; 268 test rows across 29 machines (zero machine overlap — true out-of-distribution evaluation).

Data & Pre-processing

Raw Data Format

Each row in `training_data.csv` / `test_data.csv` represents one decision point during a pentest session:

Column	Description
`Machine`	Target machine name (e.g. "bashed", "Beep")
`PTT`	Full Penetration Testing Tree — indented text tracking completed/in-progress tasks
`Previous strategy`	The strategic goal before this step
`Previous step`	What was attempted last
`Previous step result`	What was found/achieved
`New strategy`	The updated strategic intent
`Strategy explanation`	Why the strategy changed
`New step`	**Gold label** — next step type (one of 10 categories)
`Step explanation`	**Gold label** — free-text reasoning
`MCP_tasks`	**Gold label** — dict of `{tool_name: action_description}`

Step 1 — Graph Construction (`generate_graphs.py`)

For each machine, the PTT text is parsed into a structured **Agent-Search-Track graph** representing the evolving recon state.

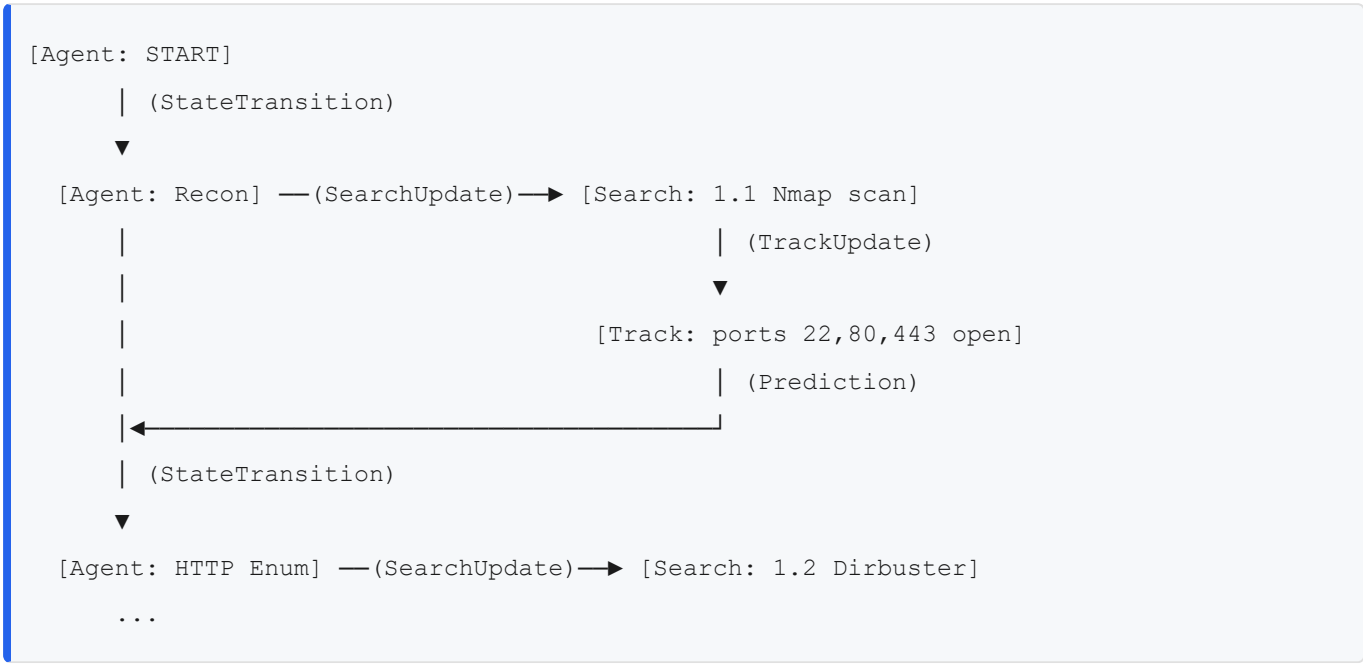
Graph node types:

Type	Color	Meaning
Agent (blue)	`#3A86FF`	Cumulative PTT state after completing a task item
Search (orange)	`#FB5607`	The specific PTT item being worked + its MCP tool calls
Track (green)	`#06D6A0`	Findings payload — what was actually discovered
Agent Goal (pink)	`#FF006E`	Final state when the pentest concludes

Edge types:

Type	Color	Direction
StateTransition	Black	Agent → Agent (advancing through PTT)
SearchUpdate	Green	Agent → Search (starting work on a PTT item)
TrackUpdate	Blue	Search → Track (item execution produced findings)
Prediction	Purple	Track → Agent (findings lead to next state)

Example graph structure for a 3-step sequence:



Output: one `<machine>_graph.json` per machine in `processed_data/train/` or `processed_data/test/`.

Step 2 — Input JSON Assembly (`build_input_json.py`)

Joins each CSV row with its machine's graph JSON into a single unified record:

```
{
    "Machine": "bashed",
    "Graph": { "nodes": [...], "edges": [...] },
    "Previous strategy": "...",
    "Previous step": "...",
    "Previous step result": "...",
    "New strategy": "...",
    "Strategy explanation": "...",
    "Gold New step": "Exploit the selected exploitations",
    "Gold Step explanation": "The previous step identified...",
    "Gold MCP_tasks": "{'Metasploit': 'Exploit php/webapps/...'}"
  }
```

Output: `input/train.json` (1,894 records) and `input/test.json` (268 records).

Stage 1 — GNN Classifier

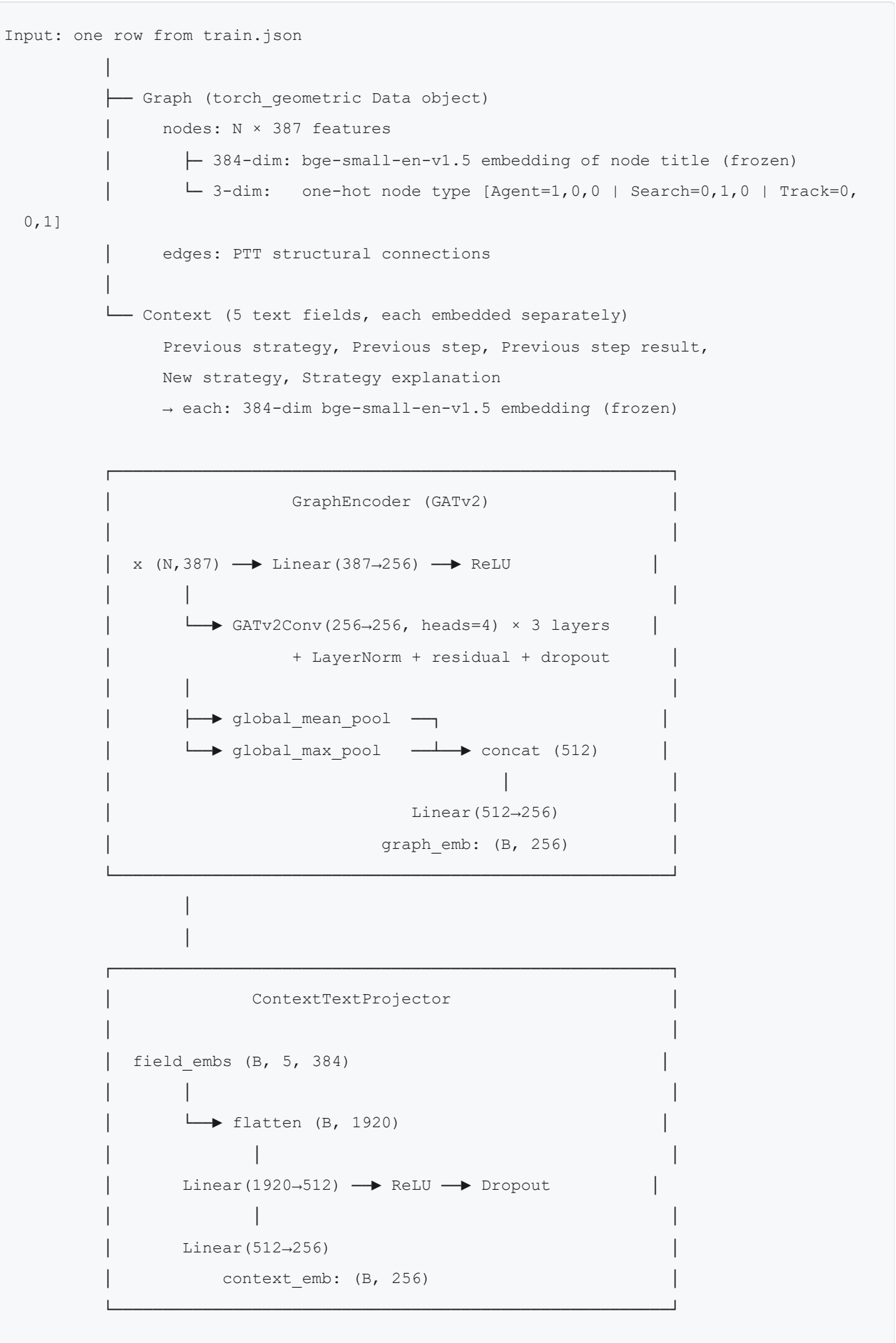
File: `stage1_gnn_train.py`, `graph_encoder.py`
Checkpoint: `checkpoints/stage1_gnn_classifier.pt`

What it does

Stage 1 trains a purely discriminative model — no text generation. Given the current graph state and context text, it classifies:

- Which of the 10 step types to take next (single-label)
- Which of the 11 MCP tools to use (multi-label, independent binary decisions)

Architecture


Input: one row from train.json
Graph (torch_geometric Data object)
nodes: N × 387 features
384-dim: bge-small-en-v1.5 embedding of node title (frozen)
3-dim: one-hot node type [Agent=1,0,0 | Search=0,1,0 | Track=0, 0,1]
edges: PTT structural connections
Context (5 text fields, each embedded separately)
Previous strategy, Previous step, Previous step result,
New strategy, Strategy explanation
→ each: 384-dim bge-small-en-v1.5 embedding (frozen)
GraphEncoder (GATv2)
x (N,387) ⟶ Linear(387→256) ⟶ ReLU
GATv2Conv(256→256, heads=4) × 3 layers
+ LayerNorm + residual + dropout
global_mean_pool
global_max_pool
concat (512)
Linear(512→256)
graph_emb: (B, 256)
ContextTextProjector
field_embs (B, 5, 384)
flatten (B, 1920)
Linear(1920→512) ⟶ ReLU ⟶ Dropout
Linear(512→256)
context_emb: (B, 256)

```
                │
                ▼
          concat([graph_emb, context_emb])  →  (B, 512)
                │
          ┌─────────────────────────┐
          │  Fusion MLP             │
          │  Linear(512→512) ReLU   │
          │  Dropout(0.1)           │
          │  Linear(512→512) ReLU   │
          └─────────────────────────┘
                │
                ├──► step_head: Linear(512→10)  →  softmax  →  step prediction
                │
                └──► mcp_head:  Linear(512→11)  →  sigmoid  →  multi-label MCP predic
  tion
```

Also outputs: `graph_emb (B, 256)` — reused by Stages 2 and 3 as the graph conditioning signal.

Training

- **Loss:** `CrossEntropy(step) + BCE(mcp)`, equal weight 1.0 each, **unweighted** (known issue)
- **Optimizer:** AdamW, lr=2e-4, weight_decay=1e-4
- **Schedule:** CosineAnnealingLR over 30 epochs
- **Batch size:** 16, with 10% held-out validation
- **Best checkpoint:** saved when `step_macro_f1 + mcp_micro_f1` is maximised on val set

Input / Output Example

Input:

```
Graph: 23 nodes (8 Agent, 8 Search, 7 Track)
         node titles: "Agent: Initial (Start)", "Search: 1.1 Nmap scan",
                      "Track: ports 22,80,443 open", ...
  Context:
    Previous strategy: "Perform active information gathering"
    Previous step: "Run Nmap full port scan"
    Previous step result: "Found nginx 1.18.0 on port 80, SSH on 22"
    New strategy: "Enumerate HTTP service for hidden directories"
    Strategy explanation: "Web service discovery needed before exploitation"
```

Output:

```
{
    "step_pred": "Enumerate further on the X service to find software versions, hidden
  directories and file.",
    "mcp_pred": ["Dirbuster", "Web page interaction"]
  }
```

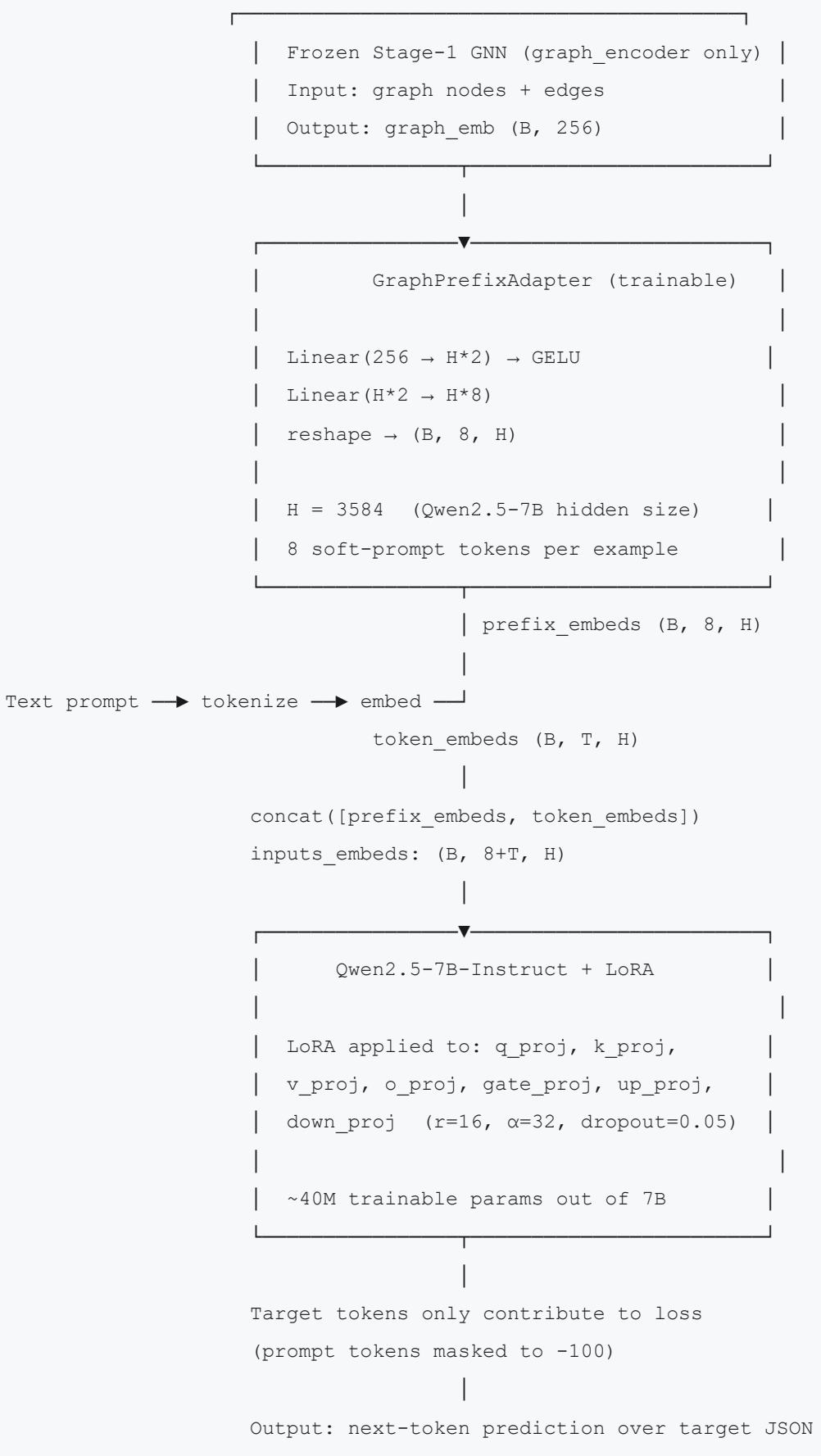
Stage 2 — Supervised Fine-Tuning (SFT) of Qwen

File: `stage2_sft_qwen.py`
Checkpoint: `checkpoints/stage2_qwen_lora/`
Base model: `Qwen/Qwen2.5-7B-Instruct`

What it does

Stage 2 takes the frozen Stage 1 GNN embedding and conditions a 7B LLM to generate a structured JSON output containing all three predictions simultaneously: step type, MCP tools, and a free-text explanation. The graph is injected as soft-prompt tokens — learned embeddings prepended to the text input.

Architecture



Prompt Format

```
<|system|>
  You are an autonomous penetration-testing planning assistant operating
  strictly within an authorized lab environment. Given the current
  reconnaissance graph state and the previous/new strategy context,
  choose exactly one next-step type from the fixed taxonomy, exactly one
  or more tool(s) from the fixed MCP taxonomy, and explain your reasoning.

  <|user|>
  Machine: bashed
  Previous strategy: Perform active information gathering
  Previous step: Run Nmap full port scan
  Previous step result: Found nginx 1.18.0 on port 80, SSH on 22
  New strategy: Enumerate HTTP service for hidden directories
  Strategy explanation: Web service discovery needed before exploitation

  <|assistant|>
```

Target (Gold) Format

```
{
    "New step": "Enumerate further on the X service to find software versions, hidden d
  irectories and file.",
    "Step explanation": "The previous step successfully identified port 80 running ngin
  x 1.18.0...",
    "MCP_tasks": {
      "Dirbuster": "Use Dirbuster as part of: Enumerate further on the X service...",
      "Web page interaction": "Use Web page interaction as part of: Enumerate furthe
  r..."
    }
  }
```

Training

- **Effective batch size:** 16 (batch=2 × grad_accum=8)
- **Optimizer:** AdamW, lr=1e-4, weight_decay=0.01
- **Schedule:** Cosine with warmup (5% of total steps)
- **Epochs:** up to 10 with early stopping (patience=3) on validation loss
- **Val split:** 10% held out
- **max_len:** 2048 tokens per example (prompt truncated to 900 to leave room for output)

Stage 3 — GRPO Reinforcement Learning

File: `stage3_grpo_rl.py`
Checkpoint: `checkpoints/stage3_qwen_grpo/`
Starts from: Stage 2 LoRA checkpoint

What it does

Stage 3 applies **Group Relative Policy Optimization (GRPO)** to refine the Stage 2 model using a fully deterministic reward signal — no LLM judge. For each training example, the model generates G=4 candidate completions, scores them against the gold labels, and uses the relative ranking within the group as the training signal.

The custom loop (not TRL's GRPOTrainer) is required because the graph soft-prompt tokens must be injected at every rollout. TRL's trainer only supports token IDs, which would drop the graph conditioning entirely during RL.

Reward Function

Each completion receives a composite score:

```
reward = 0.10 × format_ok
       + 0.30 × step_exact_match
       + 0.30 × mcp_set_F1
       + 0.30 × explanation_bertscore
```

Component	Score	Description
`format_ok`	0 or 1	Valid JSON with all 3 required keys present
`step_exact_match`	0 or 1	`"New step"` exactly matches gold label string
`mcp_set_F1`	0.0–1.0	F1 between predicted and gold tool sets
`explanation_bertscore`	0.0–1.0	Cosine similarity between bge-small-en-v1.5 embeddings of predicted and gold explanation (clamped to 0 if < 0.30 threshold)

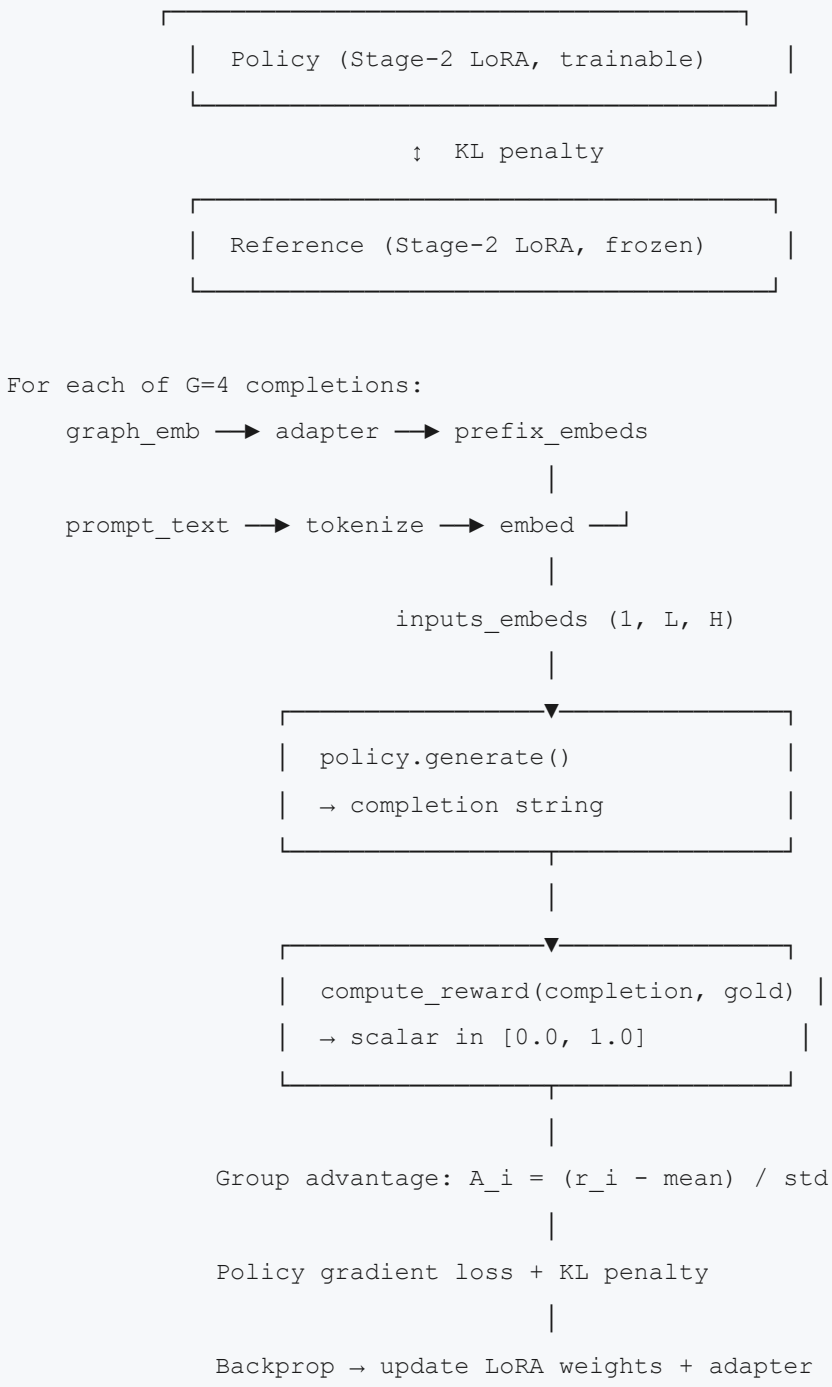
GRPO Training Loop

```
For each training step (1 to 1000):
      1. Sample one example from training set
      2. Build graph prefix embeddings (frozen GNN + trainable adapter)
      3. Tokenize and embed the text prompt
      4. inputs_embeds = concat(prefix, token_embeds)
      5. Generate G=4 completions (temperature=0.8, top_p=0.9)
      6. Decode and score each completion → rewards r_1..r_G
      7. Group-relative advantage: A_i = (r_i - mean(r)) / (std(r) + ε)
      8. For each completion g:
           policy_logprob  = log π_θ(completion | prefix+prompt)
           ref_logprob     = log π_ref(completion | prefix+prompt)  [frozen Stage 2]
           KL_penalty      = policy_logprob - ref_logprob
           pg_loss_g       = -A_g × policy_logprob + β × KL_penalty
      9. total_loss = mean(pg_loss_g for g in G) / grad_accum
     10. Backward; optimizer step every 8 steps
```

Hyperparameters:

- Group size G = 4
- KL coefficient β = 0.02
- LR = 5e-6
- Total steps = 1000 (effective: ~2 passes over training data)
- Gradient accumulation = 8

Architecture Flow



Evaluation

File: `evaluate.py`
Test set: `input/test.json` — 268 rows, 29 machines (zero overlap with training machines)

All three models are evaluated on the same test set. The GNN (Stage 1) is evaluated as a classifier only; Stages 2 and 3 generate text which is then parsed and evaluated.

Metrics — Step Classification

- **Accuracy:** fraction of examples where predicted step exactly matches gold label
- **Macro F1:** unweighted average F1 across all 10 classes — penalises poor performance on rare classes equally
- **Weighted F1:** class-frequency-weighted average — closer to what accuracy measures

Metrics — MCP Tool Classification (multi-label)

- **Subset (exact-match) accuracy:** fraction where predicted tool set exactly equals gold tool set
- **Micro F1:** aggregate TP/FP/FN across all labels — dominated by frequent tools
- **Macro F1:** unweighted average F1 per label — rare tools (SQLmap, hydra) drag this down
- **Samples F1:** average per-sample F1 — handles partial overlaps

Metrics — Explanation Quality (Stages 2 & 3 only)

Metric	What it measures
BLEU-1/2/4	N-gram overlap between generated and gold explanation
ROUGE-L	Longest common subsequence between generated and gold
BERTScore-F1	Semantic similarity via bge-small-en-v1.5 cosine similarity
Step Alignment	Fraction of explanations that mention keywords from the predicted step label
Context Grounding	Fraction that reference tokens from the previous step result
Reasoning Density	Fraction of causal/justification language words present
Empty rate	Fraction of explanations that are blank (JSON parse failure indicator)

LLM Inference for Evaluation

```
For each test example:
      1. Build graph prefix via frozen GNN + loaded adapter
      2. Tokenize prompt (max 900 tokens, truncated)
      3. inputs_embeds = concat(prefix, token_embeds)
      4. model.generate(max_new_tokens=200, do_sample=False, repetition_penalty=1.1)
      5. Decode output text
      6. Parse JSON: find first "{...}" block
         → if parse fails: regex fallback for "New step" field
      7. Normalize predicted step string → match to STEP_LABELS taxonomy
      8. Extract MCP tool names from "MCP_tasks" dict keys
      9. Record step prediction, MCP prediction, explanation text
```

MCP Threshold Decision

Each MCP label uses a per-label sigmoid threshold. The Stage 1 GNN checkpoint can store calibrated per-label thresholds (from `mcp_threshold_search.py`). If none are saved, the evaluator falls back to uniform 0.5 for all labels ("Legacy checkpoint" warning). The LLM stages use text parsing, not sigmoid thresholds, for MCP prediction.

Results

Run date: July 2026 | **Total pipeline time:** 2h 35m 3s | **Test set:** 268 examples, 29 machines

---

Summary Comparison

Metric	Stage 1 GNN	Stage 2 SFT Qwen	Stage 3 GRPO
Step Accuracy	**0.7351**	0.6381	0.6418
Step Macro F1	0.5166	0.5296	**0.5521**
Step Weighted F1	**0.7292**	0.6926	0.7012
MCP Subset Accuracy	**0.4627**	0.4179	0.4254
MCP Micro F1	**0.6555**	0.5971	0.6062
MCP Macro F1	0.3692	0.4224	**0.4236**
BERTScore-F1	N/A	0.6911	**0.6920**
Explanation Empty Rate	N/A	32.8%	33.2%

Key observation: Stage 1 GNN is the strongest overall step classifier. Stage 2/3 improve macro F1 and MCP macro F1 (better on rare classes) but lose overall accuracy — the 33% JSON parse failure rate in LLM stages suppresses their numbers.

Stage 1 GNN — Detailed Results

Step Classification (268 test examples)

```
Accuracy    : 0.7351
  Macro F1    : 0.5166
  Weighted F1 : 0.7292

  Per-class breakdown:
    Class                                                P      R     F1   Support
    Do a google search for more information            0.79   0.50   0.61      22
    Enumerate further on the X service...              0.86   0.81   0.83      59  ←
  best
    Explore the suspicious files, commands...          0.53   0.60   0.56      30
    Further Enumerate the website...                   0.79   0.70   0.75      27
    Enumerate the domain                               0.33   0.14   0.20       7  ←
  sparse
    Exploit the selected exploitations                 0.75   0.86   0.80      92
    Analyze the outcomes... find an attack path        0.00   0.00   0.00       3  ←
  3 test samples
    Ask for human assistant                               —      —      —       0
  (no test samples)
    Explore the source code for vulnerabilities        0.00   0.00   0.00       5  ←
  5 test samples
    End task and ask permission to generate the report 0.88   0.91   0.89      23  ←
  best terminal
```

Confusion matrix analysis:

- "Exploit" absorbs misclassifications from most other classes (gravity of the majority class, 35.4% of training data)
- "Explore suspicious files" → confused with "Exploit" (8×) and "Enumerate service" (1×)
- "Google search" → classified correctly only 50% of the time (11/22)

MCP Tool Classification

```
Subset (exact-match) accuracy : 0.4627
  Micro F1                      : 0.6555
  Macro F1                      : 0.3692
  Samples F1                    : 0.6461

  Per-label:
    Nmap               P=0.711  R=0.964  F1=0.818   (28 support) ← strong
    Interactive CLI    P=0.873  R=0.780  F1=0.824  (159 support) ← dominant class
    Dirbuster          P=0.643  R=0.783  F1=0.706   (23 support)
    Metasploit         P=0.364  R=0.706  F1=0.480   (17 support)
    Google search      P=0.733  R=0.393  F1=0.512   (28 support)
    Web page interact P=0.636  R=0.375  F1=0.472   (56 support)
    Netcat             P=1.000  R=0.143  F1=0.250   (14 support) ← high P, zero R
    SQLmap             P=0.000  R=0.000  F1=0.000    (7 support) ← never predicted
    Smb client         P=0.000  R=0.000  F1=0.000   (18 support) ← never predicted
    hydra              P=0.000  R=0.000  F1=0.000    (3 support) ← never predicted
    John-the-ripper    P=0.000  R=0.000  F1=0.000   (12 support) ← never predicted

  Note: Uniform threshold 0.5 used (legacy checkpoint — per-label calibration not yet r
  un)
```

Stage 2 SFT Qwen — Detailed Results

Inference: 268 samples × ~3.95s/sample = 17m 39s total
Parse failures: 93/268 responses (34.7%) had no valid JSON — regex fallback applied

Step Classification

```
Accuracy    : 0.6381
  Macro F1    : 0.5296
  Weighted F1 : 0.6926

  Per-class breakdown:
    Class                                                P      R     F1   Support
    Do a google search for more information            0.84   0.73   0.78      22  ←
  improved vs GNN
    Enumerate further on the X service...              0.78   0.85   0.81      59
    Explore the suspicious files, commands...          0.38   0.17   0.23      30  ←
  degraded
    Further Enumerate the website...                   1.00   0.52   0.68      27
  (high P, low R)
    Enumerate the domain                               0.17   0.14   0.15       7  ←
  degraded
    Exploit the selected exploitations                 0.87   0.67   0.76      92  ←
  degraded
    Analyze the outcomes... find an attack path        0.50   0.33   0.40       3  ←
  appeared (was 0.00)
    Ask for human assistant                               —      —      —       0
    Explore the source code for vulnerabilities        0.50   0.80   0.62       5  ←
  appeared (was 0.00)
    End task and ask permission to generate the report 0.95   0.78   0.86      23
```

The LLM recovers some rare class recall ("Analyze outcomes", "Explore source code") but loses accuracy on frequent classes due to JSON parse failures pushing those to -1 (unknown) predictions.

MCP Tool Classification

```
Subset accuracy : 0.4179   (worse than GNN)
  Micro F1      : 0.5971   (worse than GNN)
  Macro F1      : 0.4224   (better than GNN — rare tools now appear)

    Nmap              P=1.000  R=0.857  F1=0.923  ← best
    John-the-ripper   P=0.556  R=0.417  F1=0.476  ← appeared
    Dirbuster         P=0.778  R=0.609  F1=0.683
    hydra             P=1.000  R=0.333  F1=0.500  ← appeared (precision from text)
    Google search     P=0.857  R=0.429  F1=0.571
    Interactive CLI   P=0.959  R=0.591  F1=0.732
    Smb client        P=1.000  R=0.167  F1=0.286  ← still low recall
    Metasploit        P=0.333  R=0.059  F1=0.100  ← degraded
    Netcat            P=0.500  R=0.071  F1=0.125  ← still low recall
    Web page interact P=1.000  R=0.143  F1=0.250  ← very low recall
    SQLmap            P=0.000  R=0.000  F1=0.000  ← still never predicted
```

Explanation Quality

```
BLEU-1          : 0.2798
  BLEU-2        : 0.1255
  BLEU-4        : 0.0457
  ROUGE-L       : 0.2214
  BERTScore-F1  : 0.6911   ← semantic similarity reasonable
  Step Alignment : 0.1216   ✗ (should be > 0.20)
  Context Ground. : 0.0780
  Reasoning Dens. : 0.0376
  Avg length    : 15.9 tokens   (gold avg: ~70 tokens)
  Empty rate    : 32.8%         ← critical issue
```

Diagnosis: The 33% empty rate is the single biggest problem. When the model produces an incomplete JSON (truncated at `max_new_tokens=200`), the explanation field is blank. The 16-token average length confirms truncation is widespread.

Stage 3 GRPO — Detailed Results

Inference: 268 samples × ~3.99s/sample = 17m 48s total
Parse failures: 92/268 responses (34.3%) — nearly identical to Stage 2

Step Classification

```
Accuracy    : 0.6418   (+0.37% vs Stage 2)
  Macro F1    : 0.5521   (+0.23 vs Stage 2)   ← measurable improvement
  Weighted F1 : 0.7012   (+0.86% vs Stage 2)

  Improvements over Stage 2:
    Enumerate domain:    0.15 → 0.29   (+0.14)
    Exploit:             0.76 → 0.77   (+0.01)
    Explore source code: 0.62 → 0.67   (+0.05)
    Further enum website:0.68 → 0.74   (+0.06)

  Regressions vs Stage 2:
    Enumerate service:   0.81 → 0.79   (-0.02)
```

MCP Tool Classification

```
Subset accuracy : 0.4254   (+0.75% vs Stage 2)
  Micro F1        : 0.6062   (+0.91% vs Stage 2)
  Macro F1        : 0.4236   (+0.12% vs Stage 2)

  Notable changes vs Stage 2:
    Web page interact: 0.250 → 0.459  (+0.21)  ← biggest gain
    Smb client:        0.286 → 0.190  (-0.10)  ← regression
    Dirbuster:         0.683 → 0.579  (-0.10)  ← regression
```

Explanation Quality

```
BLEU-1          : 0.2785   (≈ Stage 2)
  BLEU-4          : 0.0572   (+0.01 vs Stage 2)
  ROUGE-L         : 0.2229   (≈ Stage 2)
  BERTScore-F1    : 0.6920   (+0.001 vs Stage 2)
  Step Alignment  : 0.1173   (slightly worse)
  Empty rate      : 33.2%    (≈ Stage 2 — still broken)
```

GRPO improvement is marginal. With only 1,000 training steps and G=4 rollouts (4,000 total completions over 1,894 examples), each example was seen on average only 2.1 times. Standard GRPO convergence requires 50–100+ passes per example. The reward signal improved some rare step/tool predictions but could not overcome the JSON truncation problem, which is a generation budget issue not addressable by RL reward shaping.
